

Contents

Bot Challenge & Discovery — Micro Plan	1
Phase A — Primitives (no user-visible surface)	1
Phase B — Directory page + first two entry points	2
Phase C — Table-create picker + Quick Match	2
Sequencing rules	3
Observability hooks (free with this layout)	3
Risks & mitigations	3
Effort summary	3

Bot Challenge & Discovery — Micro Plan

Goal. Make it one click to challenge any bot — your own, a community bot, or another player’s bot — from anywhere a bot is visible. Build a small reusable primitive layer first, then layer on five UX entry points that all share the same code path.

Why now. Playing against other players’ bots is a key early-platform feature; the backend already supports H-vs-any-bot via `POST /rt/tables { kind: 'hvb', botUserId }` and `GET /api/v1/bots` already lists every active bot publicly. The gap is purely UX-side discovery and one-click action.

Out of scope. Spectator-of-bot-vs-bot (already shipped). Tournament bot pickers (already shipped). Bot-vs-bot quick-match (already shipped via Spar). Auto-matchmaking queues (separate, larger initiative).

Cross-cutting design decision — guest-friendly by default. Every challenge surface in this plan (`<ChallengeButton>`, the `/bots` directory, profile / rankings / picker / Quick Match) is reachable and usable by unauthenticated visitors. Rationale: the deeper a guest gets into actual gameplay, the higher the conversion to a long-term registered player — and the platform already establishes this pattern with `/play?action=vs-community-bot` (open to guests today) and the post-game guest-signup CTA in `PlayPage.jsx`. Sign-in is asked for *after* the user has felt value, not as a gate to it. Concrete implications: - `<ChallengeButton>` does not check auth before posting — anonymous SSE sessions already have a synthesised seat id, and `createHvbTable` accepts that path today. - `GET /api/v1/bots/quick-match` accepts anonymous callers; for guests the ELO window is centered on a default rating (1500) since they have no `GameElo` row yet. - The directory page renders identically to guests, except a “Sign in to save your stats” footer ribbon is shown (non-blocking). - Existing post-game signup CTA on `PlayPage` continues to fire as today — that’s the natural conversion moment.

Phase A — Primitives (no user-visible surface)

Build four reusable pieces that all later phases consume. Land with unit tests; no UI integration yet.

Primitive	Location	Surface area
<code>useBots({ game, eloMin, eloMax, owner, search })</code>	<code>landing/src/lib/useBots.js</code>	SWR hook over <code>GET /api/v1/bots</code> . Returns <code>{ bots, isLoading, error }</code> . Filters apply client-side over the cached bot list.
<code><BotCard bot variant /></code>	<code>landing/src/components/bots/BotCard.js</code>	Card layout: avatar, displayName, owner badge, game-specific ELO, win-rate, last-active. <code>variant</code> <code>□ 'list' \</code> <code>'select' \</code> <code>'profile'</code> controls trailing actions. Renders a children-passed action slot.
<code><ChallengeButton botUserId game /></code>	<code>landing/src/components/bots/ChallengeButton.js</code>	Stateless one-click: <code>POST /rt/tables { kind: 'hvb', botUserId, gameId }</code> → <code>navigate('/play?join=<slug>')</code> . Disabled state, error toast, never opens a modal.
<code><BotFilterBar value onChange /></code>	<code>landing/src/components/bots/BotFilterBar.js</code>	Search input, ELO slider/range, game tab, owner toggle (mine / community / everyone). Controlled component. URL-param sync is the consumer’s responsibility.

Tests (vitest, landing-host):

- useBots — fetches once per filter combo, dedupes, surfaces error states, refetches on filter change.
- <BotCard /> — renders all variants; truncates long names; handles missing ELO.
- <ChallengeButton /> — happy-path POST + navigate; error path keeps user on page; disabled while in-flight.
- <BotFilterBar /> — emits change events for each control; clears reset.

Exit criteria: all 4 modules exported and unit-tested. No imports from outside landing/src/components/bots/ and landing/src/lib/useBots.js. Land in a single /dev cycle.

Effort: ~half day.

Phase B — Directory page + first two entry points

Three surfaces, all consuming Phase A primitives. Ship in one /dev cycle since they share a single test pass.

B.1 — Bot Directory page (/bots)

- New landing/src/pages/BotDirectoryPage.jsx
- Composition: <BotFilterBar /> + grid of <BotCard variant="list"> with <ChallengeButton /> in the action slot
- URL params drive filter state for shareable filtered views (/bots?game=xo&eloMin=1200)
- Empty/loading/error states

B.1.x — Primary nav update (decided)

- Edit packages/nav/src/navItems.js: replace the about entry with bots (label “Bots”, to “/bots”, app “landing”). Final order: **Tables · Tournaments · Gym · Rankings · Bots · Profile** (6 items, under the 5-7 soft ceiling).
- Move the About link to the site footer (most apps put About / Privacy / Terms there). Footer placement TBD — pair with whatever footer/help-links surface already exists, or add one if none.
- Why: About is one-time-read material (low frequency, narrow audience). Bots is daily-use, action-oriented, broad-audience, and onboarding-critical. Swapping them keeps nav slot count flat while replacing a low-value item with a high-value one. Decision rationale: passes all five nav-placement tests (5-7 rule, frequency × audience, conceptual distinctness, 1-click test, mobile 5-slot fit).
- Update any tests / e2e specs that hard-code “About” as a primary-nav item.

B.2 — Profile page “Challenge” button (#1 from the brainstorm)

- landing/src/pages/BotProfilePage.jsx: drop <ChallengeButton /> next to the existing Spar button
- ~10 lines

B.3 — Rankings row challenge icon (#2 from the brainstorm)

- landing/src/pages/RankingsPage.jsx: render <ChallengeButton variant="icon" /> in each bot row
- Add a compact variant="icon" mode to <ChallengeButton /> (just the  / “Play” mini-button)
- ~10 lines + ~10 lines on the button

Exit criteria: /bots, /bots/:id Challenge button, and /rankings row challenge icons all working end-to-end against a real backend. New e2e spec bot-challenge-flow.spec.js: from each entry point, one click → arrives on /play?join=<slug> with both seats populated.

Effort: ~half day.

Phase C — Table-create picker + Quick Match

Two independent scope items, each its own /dev cycle. Order doesn’t matter.

C.1 — Expand the create-table bot picker (#3)

- landing/src/components/tables/CreateTableForm.jsx (or wherever the current bot dropdown lives — confirm path before starting)
- Replace the owner-scoped dropdown with a tabbed picker:

- **My Bots** (default) — current behaviour
- **Community** — built-in / system bots
- **All Bots** — every active bot
- Reuse `<BotFilterBar /> + <BotCard variant="select">`. The select variant emits `onSelect(botUserId)` instead of rendering a `<ChallengeButton />`
- Existing tests for create-table flow continue to pass (selection mechanism changes, the seat-claim path doesn't)

Effort: ~3-4 hours.

C.2 — Quick Match button on home (#4)

- New server route `GET /api/v1/bots/quick-match?eloWindow=100&gameId=xo` — picks a random active bot near the caller's ELO (defaults: ± 100 rating, requires `caller.botActive=true`-equivalent for guests). Returns `{ botUserId, displayName }`
- New "Quick Match" CTA on `HomePage.jsx`: calls the route, then chains into the same `<ChallengeButton />` action
- Backend test: ELO window correctness, no-bots-available 404, anonymous caller path
- Client test: CTA → POST → navigate

Effort: ~1 day.

Sequencing rules

1. Phase A must land before any of Phase B / C (everything depends on the primitives).
2. Phase B is one /dev cycle (B.1 + B.2 + B.3 together) — same primitives, same test pass.
3. Phase C items are independent — ship in any order after B is in.
4. Each phase ends with full backend + landing test pass and the new e2e specs green.

Observability hooks (free with this layout)

Since every challenge flows through `<ChallengeButton />`, we get these metrics for free by adding one fetch tag:

- **Challenge attempt rate** by entry point (profile, rankings, directory, picker, quick-match) — tells us which surface drives most plays
- **Challenge → game-start conversion** — does the user actually finish a game after the click?
- **Time-to-first-move per entry point** — UX latency per surface

Add a single `analytics.track('bot.challenge.click', { source })` inside `<ChallengeButton />` and the metrics fall out without per-surface instrumentation.

Risks & mitigations

Risk	Mitigation
Existing <code>/play?action=vs-community-bot</code> flow drifts from <code><ChallengeButton /></code> <code>useBots()</code> over-fetches on every filter change	After Phase B, refactor that entry point to call <code><ChallengeButton /></code> with the resolved community-bot id. Single code path. Filter client-side over the cached list; don't re-hit the API for filter changes. The <code>GET /api/v1/bots</code> endpoint is already cached server-side.
Quick Match can pair you with your own bot	Server route excludes <code>botOwnerId === caller.id</code> from the candidate pool.
Rankings page already has many actions per row → "Play" button competes for space	Use an icon-only <code>variant="icon"</code> button (1.4rem+ per the icon-sizing memory). Group with existing per-row actions.
Removing About from primary nav strands a discovery path	About link moves to the footer in the same change. Audit any onboarding tour / help text that references "click About in the top nav" and update copy.
Guest path silently breaks (e.g. SSE session not minting a <code>seatId</code> for anonymous callers)	Add e2e coverage for the guest-flow case in every challenge surface — open the page with no cookies, click challenge, assert game starts. The "guest-friendly by default" decision (above) is load-bearing — protect it with tests.

Effort summary

Phase	Effort
A — Primitives	~half day
B — Directory + Profile + Rankings	~half day
C.1 — Create-table picker	~3-4 hours
C.2 — Quick Match	~1 day
Total	~3 days end-to-end